

**Федеральное государственное бюджетное образовательное учреждение
высшего образования
«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА
И ГОСУДАРСТВЕННОЙ СЛУЖБЫ
ПРИ ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»**

НИЖЕГОРОДСКИЙ ИНСТИТУТ УПРАВЛЕНИЯ – филиал РАНХиГС

**Факультет управления
Кафедра информатики и информационных технологий**

**Задания практические по дисциплине «База данных» для
очно- заочной формы обучения**

Тема 6 Запросы ОПК-2.2

Вопросы для опроса

1. Понятие запроса.
2. DDL-запросы.
3. DML-запросы.
4. Представление, как один из видов запроса.
5. Триггер, как один из видов запроса.
6. Процедура, как один из видов запроса.

Вариант 1

Этот запрос выбирает из таблицы «Билеты» (tickets) всех пассажиров с именами, состоящими из трёх букв (в шаблоне присутствуют три символа «_»):

ВЫБРАТЬ имя_пассажира

ИЗ базы_данных_о_пассажирах

ГДЕ имя_пассажира ПОДОБНО '___ %';

Предложите шаблон поиска в операторе LIKE для выбора из этой таблицы всех пассажиров с фамилиями, состоящими из пяти букв.

Вариант 2

Самые крупные самолеты в нашей авиакомпании — это Boeing 777-300. Выяснить, между какими парами городов они летают, поможет запрос:

```
SELECT DISTINCT departure_city, arrival_city
FROM routes r
      JOIN aircrafts a
      ON r.aircraft_code = a.aircraft_code
WHERE a.model = 'Boeing 777-300'
ORDER BY 1;
```

Модифицируйте запрос таким образом, чтобы каждая пара городов была выведена только один раз.

Вариант 3

Для ответа на вопрос, сколько рейсов выполняется из Москвы в Санкт-Петербург, можно написать совсем простой запрос:

```
SELECT count(*)
FROM routes
WHERE departure_city = 'Москва'
AND arrival_city = 'Санкт-Петербург'
```

А с помощью какого запроса можно получить результат в таком виде?

departure_city	arrival_city	count
Москва	Санкт-Петербург	12

Вариант 4

Ответить на вопрос о том, каковы максимальные и минимальные цены билетов на все направления, может такой запрос:

```
SELECT
    f.departure_city,
    f.arrival_city,
    max(tf.amount),
    min(tf.amount)
FROM flights_v f
JOIN ticket_flights tf
    ON f.flight_id = tf.flight_id
GROUP BY 1, 2
ORDER BY 1, 2;
```

А как выявить те направления, на которые не было продано ни одного билета?

Вариант 5

В 4 вопросе лекции мы использовали рекурсивный алгоритм в общем табличном выражении. Изучите этот пример, чтобы лучше понять работу рекурсивного алгоритма:

```
WITH RECURSIVE ranges (min_sum, max_sum)
AS (
    VALUES (0, 100000),
            (100000, 200000),
            (200000, 300000)
    UNION ALL
    SELECT min_sum + 100000, max_sum + 100000
    FROM ranges
    WHERE max_sum < (
        SELECT max( total_amount )
        FROM bookings
    )
)
SELECT * FROM ranges;
```

Вариант 5.1

Модифицируйте запрос, добавив в него столбец level (можно назвать его и iteration). Этот столбец должен содержать номер текущей итерации, поэтому нужно увеличивать его значение на единицу на каждом шаге. Не забудьте задать начальное значение для добавленного столбца в предложении VALUES.

Вариант 5.2

Для завершения экспериментов замените UNION ALL на UNION и выполните запрос. Сравните этот результат с предыдущим, когда мы

использовали UNION ALL.

```
WITH RECURSIVE ranges (min_sum, max_sum)
AS (
  VALUES (0, 100000),
          (100000, 200000),
          (200000, 300000)
  UNION
  SELECT min_sum + 100000, max_sum + 100000
  FROM ranges
  WHERE max_sum < (
    SELECT max( total_amount )
    FROM bookings
  )
)
SELECT * FROM ranges;
```

```
SELECT city
FROM airports
WHERE city <> 'Москва'
?
SELECT arrival_city
FROM routes
WHERE departure_city = 'Москва'
ORDER BY city;
```

Вариант 7

На лекции мы рассматривали такой запрос: получить перечень аэропортов в тех городах, в которых больше одного аэропорта.

```
SELECT
  aa.city,
  aa.airport_code,
  aa.airport_name
FROM (
  SELECT
    city,
    count(*)
  FROM airports
  GROUP BY city
  HAVING count(*) > 1
) AS a
JOIN airports AS aa
  ON a.city = aa.city
ORDER BY aa.city, aa.airport_name;
```

Как вы думаете, обязательно ли наличие функции count в подзапросе в предложении SELECT или можно написать просто SELECT city FROM airports?

Вариант 8

Предположим, что департамент развития нашей авиакомпании задался вопросом: каким будет общее число различных маршрутов, которые теоретически можно проложить между всеми городами? Если в каком-то городе имеется более одного аэропорта, то это учитывать не будем, т. е. маршрутом будем считать путь между городами, а не между аэропортами. Здесь мы используем соединение таблицы с самой собой на основе неравенства значений атрибутов.

```
SELECT count( * )  
FROM ( SELECT DISTINCT city FROM airports ) AS a1  
JOIN ( SELECT DISTINCT city FROM airports ) AS a2  
ON a1.city <> a2.city;
```

Перепишите этот запрос с общим табличным выражением.

Задания для самостоятельного выполнения

1. Выполнить запрос: каковы максимальные и минимальные цены билетов на все направления?
2. Выполнить запрос: выявить те направления, на которые не было продано ни одного билета?
3. Выполнить запрос: как часто встречаются различные имена среди пассажиров?
4. Выполнить запрос: для ответа на вопрос частота распределения фамилий пассажиров.
5. Выполнить запрос: в какой ситуации, связанной с базой данных «Авиаперевозки», было бы полезно применить оконные функции, и напишите запрос.
6. Напишите запрос с использованием предложения FILTER с агрегатной функцией.
7. Выполнить запрос: как распределяются места с разными классами обслуживания в самолетах всех типов?
8. Выполнить запрос: сколько маршрутов обслуживают самолеты каждого типа?
9. Модифицируйте запрос, добавив в него столбец level (можно назвать его и iteration). Этот столбец должен содержать номер текущей итерации, поэтому нужно увеличивать его значение на единицу на каждом шаге. Не забудьте задать начальное значение для добавленного столбца в предложении VALUES.
10. Для завершения экспериментов замените UNION ALL на UNION и выполните запрос. Сравните этот результат с предыдущим, когда

выполнили запрос и использовали UNION ALL, вычисляющий распределение сумм бронирований по диапазонам в 100 тысяч рублей.

11. Выполнить запрос: выводящий список городов, в которые нет рейсов из Москвы (используется одна из операций над множествами строк: объединение, пересечение или разность)

12. Выполнить запрос: получить перечень аэропортов в тех городах, в которых больше одного аэропорта.

13. Выполнить запрос: каким будет общее число различных маршрутов, которые теоретически можно проложить между всеми городами? Если в каком-то городе имеется более одного аэропорта, то это учитывать не будем, т. е. маршрутом будем считать путь между городами, а не между аэропортами.

14. Выполнить запрос: выбирающих аэропорты в часовых поясах Asia/Novokuznetsk и Asia/Krasnoyarsk.

15. Выполнить запрос: подсчитывающий количество маршрутов, проложенных из самых восточных аэропортов.

16. Запрос какова усредненная степень заполнения самолетов на всех направлениях.

17. Выполнить запрос: чтобы он выводил те же отчетные данные, но с учетом классов обслуживания, т. е. Business, Comfort и Economy.

18. Выполнить запрос: о размещении пассажиров одного из рейсов Кемерово — Москва в салоне самолета. (Выберем для дальнейшей работы рейс, у которого значения атрибутов flight_id — 27584, aircraft_code — SU9.

19. Выполнить запрос: получить список пассажиров этого рейса с местами, которые им были назначены в салоне самолет, отсортировать строки по фамилиям пассажиров, незанятые места также должны быть выведены (поэтому используем левое внешнее соединение LEFT OUTER JOIN), отсортировать места в порядке их расположения в салоне самолета и вывести также адреса электронной почты пассажиров (у кого они были указаны при бронировании). Для выполнения второго требования надо воспользоваться столбцом contact_data. В нем содержатся JSON-объекты, содержащие контактные данные пассажиров. Ряд из них имеет ключ email.) Перепишите последний запрос с использованием выражения и добавьте столбец «Класс обслуживания» (fare_conditions)

Тема 7 Изменение данных ОПК-2.2

Вопросы для опроса

1. Вставка строк в таблицы.
2. Обновление строк в таблицах.
3. Удаление строк из таблиц.
4. Перечень основных объектов баз данных.
5. Оператор CREATE.
6. Оператор ALTER.

Вариант 1

Добавьте в определение таблицы `aircrafts_log` значение по умолчанию `current_timestamp` и соответствующим образом измените команды `INSERT`

```
-- DROP TABLE aircrafts_log;  
-- TRUNCATE TABLE aircrafts_tmp;  
CREATE TEMP TABLE aircrafts_log AS  
SELECT * FROM aircrafts WITH NO DATA;  
ALTER TABLE aircrafts_log  
ADD COLUMN log_timestamp TIMESTAMP DEFAULT (current_timestamp);
```

Вариант 2

В предложении `RETURNING` можно указывать не только символ «*», означающий выбор всех столбцов таблицы, но и более сложные выражения, сформированные на основе этих столбцов. В тексте главы мы копировали содержимое таблицы «Самолеты» в таблицу `aircrafts_tmp`, используя в предложении `RETURNING` именно «*». Однако возможен и другой вариант запроса:

```
WITH add_row AS  
( INSERT INTO aircrafts_tmp  
  SELECT * FROM aircrafts  
  RETURNING aircraft_code, model, range,  
            current_timestamp, 'INSERT'  
  )  
INSERT INTO aircrafts_log  
SELECT ? FROM add_row;
```

Что нужно написать в этом запросе вместо вопросительного знака?

Вариант 3

В предложениях `ON CONFLICT` команды `INSERT` мы использовали только выражения, состоящие из имени одного столбца. Однако в таблице «Места» (`seats`) первичный ключ является составным и включает два столбца. Напишите команду `INSERT` для вставки новой строки в эту таблицу и предусмотрите возможный конфликт добавляемой строки со строкой, уже имеющейся в таблице. Сделайте два варианта предложения `ON CONFLICT`: первый — с использованием перечисления имен столбцов для проверки наличия дублирования, второй — с использованием предложения `ON CONSTRAINT`. Для того чтобы не изменить содержимое таблицы «Места», создайте ее копию и выполняйте все эти эксперименты с таблицей-копией.

Задания для самостоятельного выполнения

1. Добавьте в определение таблицы `aircrafts_log` значение по умолчанию `current_timestamp` и соответствующим образом измените команды `INSERT`. В предложении `RETURNING` можно указывать не только символ «*», означающий выбор всех столбцов таблицы, но и более сложные выражения, сформированные на основе этих столбцов. В тексте копировали содержимое таблицы «Самолеты» в таблицу `aircrafts_tmp`, используя в

предложении RETURNING именно «*». Однако возможен и другой вариант запроса:

```
WITH add_row AS
( INSERT INTO aircrafts_tmp
  SELECT * FROM aircrafts
  RETURNING aircraft_code, model, range,
  current_timestamp, 'INSERT'
)
INSERT INTO aircrafts_log
SELECT ? FROM add_row;
```

Что нужно написать в этом запросе вместо вопросительного знака?

Приведите примеры использования используя базу данных.

2. Если бы мы для копирования данных в таблицу aircrafts_tmp использовали команду INSERT без общего табличного выражения INSERT INTO aircrafts_tmp SELECT * FROM aircrafts; то в качестве выходного результата мы увидели бы сообщение INSERT 09. Как вы думаете, что будет выведено, если дополнить команду предложением RETURNING *? INSERT INTO aircrafts_tmp SELECT * FROM aircrafts RETURNING *; Проверьте ваши предположения на практике. Подумайте, каким образом можно использовать выведенный результат? Приведите примеры использования используя базу данных.

3. В предложениях ON CONFLICT команды INSERT мы использовали только выражения, состоящие из имени одного столбца. Однако в таблице «Места» (seats) первичный ключ является составным и включает два столбца. Напишите команду INSERT для вставки новой строки в эту таблицу и предусмотрите возможный конфликт добавляемой строки со строкой, уже имеющейся в таблице. Сделайте два варианта предложения ON CONFLICT: первый — с использованием перечисления имен столбцов для проверки наличия дублирования, второй — с использованием предложения ON CONSTRAINT. Для того чтобы не изменить содержимое таблицы «Места», создайте ее копию и выполняйте все эти эксперименты с таблицей-копией.

4. В предложении DO UPDATE команды INSERT может использоваться и условие WHERE. Самостоятельно ознакомьтесь с этой возможностью с помощью документации и напишите такую команду INSERT. Приведите примеры использования используя базу данных.

5. Команда COPY по умолчанию ожидает получения вводимых данных в формате text, когда значения данных разделяются символами табуляции. Однако можно представлять входные данные в формате CSV (Comma Separated Values), т. е. использовать в качестве разделителя запятую.

```
COPY aircrafts_tmp FROM STDIN WITH ( FORMAT csv );
```

Вводите данные для копирования, разделяя строки переводом строки.

Закончите ввод строкой '\.'

```
IL9, Ilyushin IL96, 9800
```

```
I93, Ilyushin IL96-300, 9800
```



```

\
COPY 2
SELECT * FROM aircrafts_tmp;
aircraft_code | model | range
-----+-----+-----

```

```

...
CN1 | Cessna 208 Caravan | 1200
CR2 | Bombardier CRJ-200 | 2700
IL9 | Ilyushin IL96 | 9800
I93 | Ilyushin IL96-300 | 9800
(11 строк)

```

Как вы думаете, почему при выводе данных из таблицы вновь введенные значения в столбце model оказались смещены вправо? Приведите примеры использования используя базу данных.

6. Команда COPY позволяет получить входные данные из файла и поместить их в таблицу. Этот файл должен быть доступен тому пользователю операционной системы, от имени которого запущен серверный процесс, как правило, это пользователь postgres.

Подготовьте файл, например, /home/postgres/aircrafts_tmp.csv, имеющий такую структуру:

- каждая строка файла соответствует одной строке таблицы aircrafts_tmp;
- значения данных в строке файла разделяются запятыми.

Например:

```

773,Boeing 777-300,11100
763,Boeing 767-300,7900
SU9,Sukhoi SuperJet-100,3000

```

7. Введите в файл данные о нескольких самолетах, причем часть из них уже должна быть представлена в таблице, а часть — нет.

Поскольку при выполнении команды COPY проверяются все ограничения целостности, наложенные на таблицу, то дублирующие строки добавлены, конечно же, не будут. А как вы думаете, строки, содержащиеся в этом же файле, но отсутствующие в таблице, будут добавлены или нет?

Проверьте свою гипотезу, выполнив вставку строк в таблицу из этого файла:

```

COPY aircrafts_tmp
FROM '/home/postgres/aircrafts_tmp.csv' WITH ( FORMAT csv );

```

8.* В тексте был приведен запрос, предназначенный для учета числа билетов, проданных по всем направлениям на текущую дату. Однако тот запрос был рассчитан на одновременное добавление только одной записи в таблицу «Перелеты» (ticket_flights_tmp). Ниже приведен более универсальный запрос, который предусматривает возможность единовременного ввода нескольких записей о перелетах, выполняемых на различных рейсах.

Пример: для проверки работоспособности предлагаемого запроса выберем несколько рейсов по маршрутам: Красноярск — Москва, Москва — Сочи, Сочи — Москва, Сочи — Красноярск. Для определения идентификаторов рейсов сформируем вспомогательный запрос, в котором даты начала и конца рассматриваемого периода времени зададим с помощью функции bookings.now. Использование этой функции необходимо, поскольку в будущих версиях базы данных могут быть представлены другие диапазоны дат.

```
SELECT flight_no, flight_id, departure_city,
arrival_city, scheduled_departure
FROM flights_v
WHERE scheduled_departure
BETWEEN bookings.now() AND bookings.now() + INTERVAL '15 days'
AND ( departure_city, arrival_city ) IN
( ( 'Красноярск', 'Москва' ),
( 'Москва', 'Сочи' ),
( 'Сочи', 'Москва' ),
( 'Сочи', 'Красноярск' )
)
ORDER BY departure_city, arrival_city, scheduled_departure;
```

Обратите внимание на предикат IN: в нем используются не индивидуальные значения, а пары значений. Предположим, что в течение указанного интервала времени пассажир планирует совершить перелеты по маршруту: Красноярск — Москва, Москва — Сочи, Сочи — Москва, Москва — Сочи, Сочи — Красноярск. Выполнив вспомогательный запрос, выберем следующие идентификаторы рейсов (в этом же порядке):

```
13829, 4728, 30523, 7757, 30829.
WITH sell_tickets AS
( INSERT INTO ticket_flights_tmp
( ticket_no, flight_id, fare_conditions, amount )
VALUES ( '1234567890123', 13829, 'Economy', 10500 ),
( '1234567890123', 4728, 'Economy', 3400 ),
( '1234567890123', 30523, 'Economy', 3400 ),
( '1234567890123', 7757, 'Economy', 3400 ),
( '1234567890123', 30829, 'Economy', 12800 )
RETURNING *
)
UPDATE tickets_directions td
SET last_ticket_time = current_timestamp,
tickets_num = tickets_num +
( SELECT count( * )
FROM sell_tickets st, flights_v f
WHERE st.flight_id = f.flight_id
AND f.departure_city = td.departure_city
AND f.arrival_city = td.arrival_city
```

```

)
WHERE ( td.departure_city, td.arrival_city ) IN
( SELECT departure_city, arrival_city
FROM flights_v
WHERE flight_id IN ( SELECT flight_id FROM sell_tickets )
);
UPDATE 4

```

В этой версии запроса предусмотрен единовременный ввод нескольких строк в таблицу `ticket_flights_tmp`, причем перелеты могут выполняться на различных рейсах. Поэтому необходимо преобразовать список идентификаторов этих рейсов в множество пар «город отправления — город прибытия», поскольку именно для таких пар и ведется подсчет числа забронированных перелетов. Эта задача решается в предложении `WHERE`, где вложенный подзапрос формирует список идентификаторов рейсов, а внешний подзапрос преобразует этот список в множество пар «город отправления — город прибытия». Затем с помощью предиката `IN` производится отбор строк таблицы `tickets_directions` для обновления. Теперь обратимся к предложению `SET`. Подзапрос с функцией `count` вычисляет количество перелетов по каждому направлению. Это коррелированный подзапрос: он выполняется для каждой строки, отобранной в предложении `WHERE`. В нем используется соединение временной таблицы `sell_tickets` с представлением `flights_v`. Это нужно для того, чтобы подсчитать все перелеты, соответствующие паре атрибутов «город отправления — город прибытия», взятых из текущей обновляемой строки таблицы `tickets_directions`. Этот подзапрос позволяет учесть такой факт: рейсы могут иметь различные идентификаторы `flight_id`, но при этом соответствовать одному и тому же направлению, а в таблице `tickets_directions` учитываются именно направления. В случае попытки повторного бронирования одного и того же перелета для данного пассажира, т. е. ввода строки с дубликатом первичного ключа, такая строка будет отвергнута, и будет сгенерировано сообщение об ошибке. В таком случае и таблица `tickets_directions` не будет обновлена. Давайте посмотрим, что изменилось в таблице `tickets_directions`.

Задание. Модифицируйте запрос и таблицу `tickets_directions` так, чтобы учет числа забронированных перелетов по различным маршрутам выполнялся для каждого класса обслуживания: `Economy`, `Business` и `Comfort`.

9.* Предположим, что руководство нашей авиакомпании решило отказаться от использования самолетов компаний `Boeing` и `Airbus`, имеющих наименьшее количество пассажирских мест в салонах. Мы должны соответствующим образом откорректировать таблицу «Самолеты» (`aircrafts_tmp`). Предлагаем такой алгоритм:

Шаг 1. Для каждой модели вычислить общее число мест в салоне.

Шаг 2. Используя оконную функцию `rank`, присвоить моделям ранги на основе числа мест (упорядочив их по возрастанию числа мест). Ранжирование выполняется в пределах каждой компании-производителя, т. е.

для Boeing и для Airbus — отдельно. Ранг, равный 1, соответствует наименьшему числу мест.

Шаг 3. Выполнить удаление тех строк из таблицы `aircrafts_tmp`, которые удовлетворяют следующим требованиям: модель — Boeing или Airbus, а число мест в салоне — минимальное из всех моделей данной компании-производителя, т. е. модель имеет ранг, равный 1

Шаг 1 выполняется в подзапросе в предложении WITH.

Шаг 2 — в главном запросе в предложении WITH.

Шаг 3 реализуется командой DELETE. Обратите внимание, что название компании-производителя мы определяем путем взятия подстроки от значения атрибута `model`: от начала строки до пробельного символа (используем функции `left` и `strpos`). Мы включили предложение RETURNING *, чтобы увидеть, какие именно модели были удалены. В реальной работе иногда возникают ситуации, когда требуется быстро заполнить таблицу тестовыми данными. В таком случае удобно воспользоваться командой INSERT с подзапросом. Конечно, число атрибутов и их типы данных в подзапросе SELECT должны быть такими, какие ожидает получить команда INSERT. Продемонстрируем такой прием на примере таблицы «Места» (`seats`). Для того чтобы выполнить команду, приведенную в этом упражнении, нужно либо сначала удалить все строки из таблицы `seats`, чтобы можно было добавлять строки в эту таблицу. Модифицируйте команду с учетом того, что в салоне бизнес-класса число мест в ряду должно быть меньше, чем в салоне экономического класса (в приведенном решении мы для упрощения задачи принимали эти числа одинаковыми). Попробуйте упростить подзапрос, отвечающий за формирование списка номеров рядов кресел:

(VALUES ('1'), ('2'), ('3'), ('4'), ('5'), ... Воспользуйтесь функцией `generate_series`

Приведите примеры использования используя базу данных.

Тема 8 Индексы ОПК-2.2

Вопросы для опроса

1. Что такое индекс?
2. Назовите общие характеристики индекса?
3. Индексы по нескольким столбцам.
4. Уникальные индексы.
5. Индексы на основе выражений.
6. Частичные индексы.

Вариант 1

Предположим, что для какой-то таблицы создан уникальный индекс по двум столбцам: `column1` и `column2`. В таблице есть строка, у которой значение атрибута `column1` равно ABC, а значение атрибута `column2` - NULL. Мы решили добавить в таблицу еще одну строку с такими же значениями ключевых атрибутов, т.е. `column1` - ABC, а `column2` - NULL.

Как вы думаете, будет ли операция вставки новой строки успешной или завершится с ошибкой? Объясните ваше решение

Вариант 2

Обратимся к таблице «Перелеты» (ticket_flights). В ней имеется столбец «Класс обслуживания» (fare_conditions), который отличается от остальных тем, что в нем могут присутствовать лишь три различных значения: Comfort, Business и Economy.

Выполните запросы, подсчитывающие количество строк, в которых атрибут fare_conditions принимает одно из трех возможных значений. Каждый из запросов выполните три-четыре раза, поскольку время может немного изменяться, и подсчитайте среднее время.

.....

В результате выполнения запроса получаем Произведем несколько запусков запроса:

1. 76 мс
2. 63 мс
3. 57 мс
4. 56 мс
5. 57 мс

Среднее время по 5 измерениям: мс.

В результате выполнения запроса получаем, время выполнения:

1. 56 мс
2. 53 мс
3. 58 мс
4. 54 мс
5. 55 мс

Среднее время по 5 измерениям: ... мс.

В результате выполнения запроса получает, время выполнения:

1. 63 мс
2. 59 мс
3. 63 мс
4. 67 мс
5. 59 мс

Среднее время по 5 измерениям:мс.

Создадим индекс к полю...

Проделаем те же эксперименты с таблицей ticket_flights. Будет ли различаться среднее время выполнения запросов для различных значений атрибута fare_conditions? Почему это имеет место?

В результате выполнения запроса получаем Произведем несколько запусков запроса:

1. 3 мс
2. 3 мс
3. 2 мс
4. 2 мс
5. 3 мс

Среднее время по 5 измерениям:... мс.

В результате выполнения запроса получаем, время выполнения:

1. 9 мс
2. 8 мс
3. 8 мс
4. 6 мс
5. 7 мс

Среднее время по 5 измерениям:....мс.

В результате выполнения запроса получаем....., время выполнения:

1. 40 мс
2. 68 мс
3. 43 мс
4. 40 мс
5. 43 мс

Среднее время выполнения по 5 наблюдениям:....мс.

Вывод:

Задания для самостоятельного выполнения

Предположим, что для какой-то таблицы создан уникальный индекс по двум столбцам: `column1` и `column2`. В таблице есть строка, у которой значение атрибута `column1` равно ABC, а значение атрибута `column2` — NULL. Решили добавить в таблицу еще одну строку с такими же значениями ключевых атрибутов, т. е. `column1` — ABC, а `column2` — NULL. Как вы думаете, будет ли операция вставки новой строки успешной или завершится с ошибкой? Объясните ваше решение. Приведите примеры использования используя базу данных.

2. В тексте шла речь о выполнении одной и той же выборки из таблицы «Билеты» (`tickets`) при наличии индекса по столбцу `passenger_name` и при его отсутствии. Вы видели, что наличие индекса ускоряет выполнение запроса почти на порядок. Если секундомер в утилите `psql` выключен, то включите его с помощью команды `\timing on` Проведите следующий эксперимент: выполните этот запрос несколько раз подряд при отсутствии индекса, а затем создайте индекс и опять выполните этот запрос несколько раз подряд.

```
SELECT count( * )
```

```
FROM tickets
```

```
WHERE passenger_name = 'IVAN IVANOV';
```

Вы увидите, что время выполнения повторных запросов к таблице сокращается, причем, когда создан индекс, оно сокращается на порядок. Как вы думаете, почему? Приведите примеры использования используя базу данных.

3. Известно, что индекс значительно ускоряет работу, если при выполнении запроса из таблицы отбирается лишь небольшая часть строк. Если же эта доля велика, скажем, половина строк или более, то большого положительного эффекта от наличия индекса уже не будет, а возможно даже, что не будет практически никакого эффекта. Наша задача — проверить это утверждение на практике. Обратимся к таблице «Перелеты» (`ticket_flights`). В

ней имеется столбец «Класс обслуживания» (fare_conditions), который отличается от остальных тем, что в нем могут присутствовать лишь три различных значения: Comfort, Business и Economy. Если секундомер в утилите psql выключен, то включите его. Выполните запросы, подсчитывающие количество строк, в которых атрибут fare_conditions принимает одно из трех возможных значений. Каждый из запросов выполните три-четыре раза, поскольку время может немного изменяться, и подсчитайте среднее время. Обратите внимание на число строк, которые возвращает функция count для каждого значения атрибута. При этом среднее время выполнения запросов для трех различных значений атрибута fare_conditions будет различаться незначительно, поскольку в каждом случае СУБД просматривает все строки таблицы.

```
SELECT count( * )  
FROM ticket_flights  
WHERE fare_conditions = 'Comfort';  
SELECT count( * )  
FROM ticket_flights  
WHERE fare_conditions = 'Business';  
SELECT count( * )  
FROM ticket_flights  
WHERE fare_conditions = 'Economy';
```

Создайте индекс по столбцу fare_conditions. Конечно, в реальной ситуации такой индекс вряд ли целесообразен, но нам он нужен для экспериментов. Проведите те же эксперименты с таблицей ticket_flights. Будет ли различаться среднее время выполнения запросов для различных значений атрибута fare_conditions? Почему это имеет место? В завершение этого упражнения отметим, что в случае ошибки планировщика при использовании индекса возможно не только отсутствие положительного эффекта, но и значительный отрицательный эффект. Приведите примеры использования используя базу данных.

4. Для одной из таблиц создайте индекс по двум столбцам, причем по одному из них укажите убывающий порядок значений столбца, а по другому — возрастающий. Значения NULL у первого столбца должны располагаться в начале, а у второго — в конце. Посмотрите полученный индекс с помощью команд psql

```
\d имя_таблицы  
\di+ имя_индекса
```

Обратите внимание, что первая команда выведет не только имя индекса, но также и имена столбцов, по которым он создан, а вторая команда выведет размер индекса. Подберите запросы, в которых созданный индекс предположительно должен использоваться, а также запросы, в которых он использоваться, по вашему мнению, не будет. Проверьте ваши гипотезы, выполнив запросы. Объясните полученные результаты.

5. В сложных базах данных целесообразно использование комбинаций индексов. Иногда бывают более полезны комбинированные индексы по

нескольким столбцам, чем отдельные индексы по единичным столбцам. В реальных ситуациях часто приходится делать выбор, т. е. находить компромисс, между, например, созданием двух индексов по каждому из двух столбцов таблицы либо созданием одного индекса по двум столбцам этой таблицы, либо созданием всех трех индексов. Выбор зависит от того, запросы какого вида будут выполняться чаще всего. Предложите какую-нибудь таблицу в базе данных «Авиаперевозки» и смоделируйте ситуации, в которых вы приняли бы одно из этих трех возможных решений. Воспользуйтесь документацией на PostgreSQL.

6. Предложите какую-нибудь таблицу в базе данных «Авиаперевозки» и смоделируйте ситуацию, в которой было бы целесообразно использование индекса на основе функции или скалярного выражения от двух или более столбцов.

7.* В лекции «Внешние ключи» говорится о том, что в некоторых ситуациях бывает целесообразно создавать индекс по столбцам внешнего ключа ссылающейся таблицы. Это позволит ускорить выполнение операций DELETE и UPDATE над главной (ссылочной) таблицей. Подумайте, есть ли такие таблицы в базе данных «Авиаперевозки», в отношении которых было бы целесообразно поступить так, как говорится в документации. Приведите примеры использования используя базу данных.

8.* В тексте главы был показан пример использования частичного индекса для таблицы «Бронирования». Для его создания мы выполняли команду

```
CREATE INDEX bookings_book_date_part_key  
ON bookings ( book_date )  
WHERE total_amount > 1000000;
```

Проведите эксперимент с целью сравнения эффекта от создания частичного индекса с эффектом от создания обычного индекса по столбцу total_amount. Для этого удалите частичный индекс, а затем создайте обычный индекс.

```
DROP INDEX bookings_book_date_part_key;  
CREATE INDEX bookings_total_amount_key  
ON bookings ( total_amount );
```

Теперь выполните тот же запрос к таблице bookings, который был приведен в тексте:

```
SELECT *  
FROM bookings  
WHERE total_amount > 1000000  
ORDER BY book_date DESC;
```

Сравните время выполнения с тем временем, которое было получено при использовании частичного индекса. Очень вероятно, что различия времени выполнения запроса будут незначительными. Самостоятельно ознакомьтесь с разделом «Частичные индексы» и попробуйте смоделировать ситуацию в предметной области «Авиаперевозки», когда частичный индекс

дал бы больший эффект, чем обычный индекс. Приведите примеры использования используя базу данных.

9. Когда выполняются запросы с поиском по шаблону LIKE или регулярными выражениями POSIX, тогда для того, чтобы использовался индекс, нужно предусмотреть следующее. Если параметры локализации системы отличаются от стандартной настройки «С» (например, «ru_RU.UTF-8»), тогда при создании индекса необходимо указать так называемый класс операторов. Существуют различные классы операторов, например, для столбца типа text это будет text_pattern_ops.

```
CREATE INDEX tickets_pass_name  
ON tickets ( passenger_name text_pattern_ops );
```

Индексы со специальными классами операторов пригодны не для всех типов запросов. Поэтому, возможно, потребуется создать еще и индекс с классом операторов по умолчанию. Самостоятельно изучите этот вопрос в «Семейства и классы операторов» Приведите примеры использования используя базу данных.

Тема 9 Транзакция. ОПК-2.2, ОПК ОС-10.2

Вопросы для опроса

1. Общая информация о транзакциях.
2. Уровень изоляции Read Uncommitted.
3. Уровень изоляции Read Committed.
4. Уровень изоляции Repeatable Read.
5. Уровень изоляции Serializable.
6. Блокировки.

Вариант 1.

Транзакции, работающие на уровне изоляции Read Committed, видят только свои собственные обновления и обновления, зафиксированные параллельными транзакциями. При этом нужно учитывать, что иногда могут возникать ситуации, которые на первый взгляд кажутся парадоксальными, но на самом деле все происходит в строгом соответствии с этим принципом.

Воспользуемся таблицей «Самолеты» (aircrafts) или ее копией. Предположим, что мы решили удалить из таблицы те модели, дальность полета которых менее 2 000 км. В таблице представлена одна такая модель — Cessna 208 Caravan, имеющая дальность полета 1 200 км. Для выполнения удаления мы организовали транзакцию. Однако параллельная транзакция, которая, причем, началась раньше, успела обновить таблицу таким образом, что дальность полета самолета Cessna 208 Caravan стала составлять 2 100 км, а вот для самолета Bombardier CRJ-200 она, напротив, уменьшилась до 1 900 км. Таким образом, в результате выполнения операций обновления в таблице по-прежнему присутствует строка, удовлетворяющая первоначальному условию, т. е. значение атрибута range у которой меньше 2000.

Наша задача: проверить, будет ли в результате выполнения двух транзакций удалена какая-либо строка из таблицы.

На первом терминале начнем транзакцию, при этом уровень изоляции Read Committed в команде указывать не будем, т. к. он принят по умолчанию:

```
BEGIN;  
SELECT *  
FROM aircrafts_tmp  
WHERE range < 2000;
```

aircraft_code	model	range
CN1	Cessna 208 Caravan	1200

(1 строка)

```
UPDATE aircrafts_tmp  
SET range = 2100  
WHERE aircraft_code = 'CN1';  
UPDATE 1  
UPDATE aircrafts_tmp  
SET range = 1900  
WHERE aircraft_code = 'CR2';  
UPDATE 1
```

На втором терминале начнем вторую транзакцию, которая и будет пытаться удалить строки, у которых значение атрибута range меньше 2000.

```
BEGIN;
```

```
BEGIN
```

```
SELECT *  
FROM aircrafts_tmp  
WHERE range < 2000;
```

aircraft_code	model	range
CN1	Cessna 208 Caravan	1200

(1 строка)

```
DELETE FROM aircrafts_tmp WHERE range < 2000;
```

Введя команду DELETE, мы видим, что она не завершается, а ожидает, когда со строки, подлежащей удалению, будет снята блокировка. Блокировка, установленная командой UPDATE в первой транзакции, снимается только при завершении транзакции, а завершение может иметь два исхода: фиксацию изменений с помощью команды COMMIT (или END) или отмену изменений с помощью команды ROLLBACK.

Давайте зафиксируем изменения, выполненные первой транзакцией. На первом терминале сделаем так:

```
COMMIT;  
COMMIT
```

Тогда на втором терминале мы получим такой результат от команды DELETE:

```
DELETE 0
```

Чем объясняется такой результат? Он кажется нелогичным: ведь команда SELECT, выполненная в этой же второй транзакции, показывала наличие строки, удовлетворяющей условию удаления.

Объяснение таково: поскольку вторая транзакция пока еще не видит изменений, произведенных в первой транзакции, то команда DELETE выбирает для удаления строку, описывающую модель Cessna 208 Caravan, однако эта строка была заблокирована в первой транзакции командой UPDATE. Эта команда изменила значение атрибута range в этой строке.

При завершении первой транзакции блокировка с этой строки снимается (со второй строки — тоже), и команда DELETE во второй транзакции получает возможность заблокировать эту строку. При этом команда DELETE данную строку перечитывает и вновь вычисляет условие WHERE применительно к ней. Однако теперь условие WHERE для данной строки уже не выполняется, следовательно, эту строку удалять нельзя. Конечно, в таблице есть теперь другая строка, для самолета Bombardier CRJ-200, удовлетворяющая условию удаления, однако повторный поиск строк, удовлетворяющих условию WHERE в команде DELETE, не производится.

В результате не удаляется ни одна строка. Таким образом, к сожалению, имеет место нарушение согласованности, которое можно объяснить деталями реализации СУБД. Завершим вторую транзакцию:

END;

COMMIT

Вот что получилось в результате:

SELECT * FROM aircrafts_tmp;

aircraft_code	model	range
773	Boeing 777-300	11100
763	Boeing 767-300	7900
SU9	Sukhoi SuperJet-100	3000
320	Airbus A320-200	5700
321	Airbus A321-200	5600
319	Airbus A319-100	6700
733	Boeing 737-300	4200
CN1	Cessna 208 Caravan	2100
CR2	Bombardier CRJ-200	1900

(9 строк)

Задание .

Модифицируйте сценарий выполнения транзакций: в первой транзакции вместо фиксации изменений выполните их отмену с помощью команды ROLLBACK и посмотрите, будет ли удалена строка и какая конкретно.

Вариант 2

Когда говорят о таком феномене, как потерянное обновление, то зачастую в качестве примера приводится операция UPDATE, в которой значение какого-то атрибута изменяется с применением одного из действий арифметики. Например:

UPDATE aircrafts_tmp

SET range = range + 200

WHERE aircraft_code = 'CR2';

При выполнении двух и более подобных обновлений в рамках параллельных транзакций, использующих, например, уровень изоляции Read Committed, будут учтены все такие изменения (что и было показано в тексте лекции). Очевидно, что потерянного обновления не происходит.

Предположим, что в одной транзакции будет просто присваиваться новое значение, например, так:

UPDATE aircrafts_tmp

SET range = 2100

WHERE aircraft_code = 'CR2';

А в параллельной транзакции будет выполняться аналогичная команда:

UPDATE aircrafts_tmp

SET range = 2500

WHERE aircraft_code = 'CR2';

Очевидно, что сохранится только одно из значений атрибута range. Можно ли говорить, что в такой ситуации имеет место потерянное обновление? Если оно имеет место, то что можно предпринять для его недопущения? Обоснуйте ваш ответ.

Задания для самостоятельного выполнения

1. По умолчанию каждая SQL-команда, выполняемая в среде `psql`, образует отдельную транзакцию с уровнем изоляции Read Committed. Поэтому в тех экспериментах, когда одна из транзакций состоит только из единственной SQL-команды, можно не выполнять команды `BEGIN` и `END`. Конечно, если каждая из параллельных транзакций состоит из единственной SQL-команды, то хотя бы для одной из транзакций придется все же выполнить и команду `BEGIN`, иначе эксперимент не получится. В тексте лекций были приведены примеры транзакций, в которых рассматривались команды `SELECT ... FOR UPDATE` и `LOCK TABLE`. Попробуйте повторить эти эксперименты с учетом описанного поведения PostgreSQL. Приведите примеры использования используя базу данных с уровнем изоляции транзакций Read Committed.

2. Транзакции, работающие на уровне изоляции Read Committed, видят только свои собственные обновления и обновления, зафиксированные параллельными транзакциями. При этом нужно учитывать, что иногда могут возникать ситуации, которые на первый взгляд кажутся парадоксальными, но на самом деле все происходит в строгом соответствии с этим принципом. Воспользуемся таблицей «Самолеты» (`aircrafts`) или ее копией. Предположим, что мы решили удалить из таблицы те модели, дальность полета которых менее 2 000 км. В таблице представлена одна такая модель — Cessna 208 Caravan, имеющая дальность полета 1 200 км. Для выполнения удаления мы организовали транзакцию. Однако параллельная транзакция, которая, причем, началась раньше, успела обновить таблицу таким образом, что дальность полета самолета Cessna 208 Caravan стала составлять 2 100 км, а вот для самолета Bombardier CRJ-200 она, напротив, уменьшилась до 1 900

км. Таким образом, в результате выполнения операций обновления в таблице по-прежнему присутствует строка, удовлетворяющая первоначальному условию, т. е. значение атрибута range у которой меньше 2000. Наша задача: проверить, будет ли в результате выполнения двух транзакций удалена какая-либо строка из таблицы. На первом терминале начнем транзакцию, при этом уровень изоляции Read Committed в команде указывать не будем, т. к. он принят по умолчанию:

```
BEGIN;
BEGIN
SELECT *
FROM aircrafts_tmp
WHERE range < 2000;
aircraft_code | model | range
-----+-----+-----
CN1 | Cessna 208 Caravan | 1200
(1 строка)
UPDATE aircrafts_tmp
SET range = 2100
WHERE aircraft_code = 'CN1';
UPDATE 1
UPDATE aircrafts_tmp
SET range = 1900
WHERE aircraft_code = 'CR2';
UPDATE 1
```

На втором терминале начнем вторую транзакцию, которая и будет пытаться удалить строки, у которых значение атрибута range меньше 2000.

```
BEGIN;
BEGIN
SELECT *
FROM aircrafts_tmp
WHERE range < 2000;
aircraft_code | model | range
-----+-----+-----
CN1 | Cessna 208 Caravan | 1200
(1 строка)
DELETE FROM aircrafts_tmp WHERE range < 2000;
```

Введя команду DELETE, мы видим, что она не завершается, а ожидает, когда со строки, подлежащей удалению, будет снята блокировка. Блокировка, установленная командой UPDATE в первой транзакции, снимается только при завершении транзакции, а завершение может иметь два исхода: фиксацию изменений с помощью команды COMMIT (или END) или отмену изменений с помощью команды ROLLBACK.

Давайте зафиксируем изменения, выполненные первой транзакцией. На первом терминале сделаем так: COMMIT; COMMIT

Тогда на втором терминале мы получим такой результат от команды DELETE: DELETE 0

Чем объясняется такой результат? Он кажется нелогичным: ведь команда SELECT, выполненная в этой же второй транзакции, показывала наличие строки, удовлетворяющей условию удаления. Объяснение таково: поскольку вторая транзакция пока еще не видит изменений, произведенных в первой транзакции, то команда DELETE выбирает для удаления строку, описывающую модель Cessna 208 Caravan, однако эта строка была заблокирована в первой транзакции командой UPDATE. Эта команда изменила значение атрибута range в этой строке. При завершении первой транзакции блокировка с этой строки снимается (со второй строки — тоже), и команда DELETE во второй транзакции получает возможность заблокировать эту строку. При этом команда DELETE данную строку перечитывает и вновь вычисляет условие WHERE применительно к ней. Однако теперь условие WHERE для данной строки уже не выполняется, следовательно, эту строку удалять нельзя. Конечно, в таблице есть теперь другая строка, для самолета Bombardier CRJ-200, удовлетворяющая условию удаления, однако повторный поиск строк, удовлетворяющих условию WHERE в команде DELETE, не производится. В результате не удаляется ни одна строка. Таким образом, к сожалению, имеет место нарушение согласованности, которое можно объяснить деталями реализации СУБД.

Завершим вторую транзакцию:

END;

COMMIT

Вот что получилось в результате:

SELECT * FROM aircrafts_tmp;

aircraft_code | model | range

-----+-----+-----

773 | Boeing 777-300 | 11100

763 | Boeing 767-300 | 7900

SU9 | Sukhoi SuperJet-100 | 3000

320 | Airbus A320-200 | 5700

321 | Airbus A321-200 | 5600

319 | Airbus A319-100 | 6700

733 | Boeing 737-300 | 4200

CN1 | Cessna 208 Caravan | 2100

CR2 | Bombardier CRJ-200 | 1900

(9 строк)

Задание. Модифицируйте сценарий выполнения транзакций: в первой транзакции вместо фиксации изменений выполните их отмену с помощью команды ROLLBACK и посмотрите, будет ли удалена строка и какая конкретно.

3.* Когда говорят о таком феномене, как потерянное обновление, то зачастую в качестве примера приводится операция UPDATE, в которой

значение какого-то атрибута изменяется с применением одного из действий арифметики. Например:

```
UPDATE aircrafts_tmp
SET range = range + 200
WHERE aircraft_code = 'CR2';
```

При выполнении двух и более подобных обновлений в рамках параллельных транзакций, использующих, например, уровень изоляции Read Committed, будут учтены все такие изменения (что и было показано в тексте главы). Очевидно, что потерянное обновление не происходит.

Предположим, что в одной транзакции будет просто присваиваться новое значение, например, так:

```
UPDATE aircrafts_tmp
SET range = 2100
WHERE aircraft_code = 'CR2';
```

А в параллельной транзакции будет выполняться аналогичная команда:

```
UPDATE aircrafts_tmp
SET range = 2500
WHERE aircraft_code = 'CR2';
```

Очевидно, что сохранится только одно из значений атрибута range. Можно ли говорить, что в такой ситуации имеет место потерянное обновление? Если оно имеет место, то что можно предпринять для его недопущения? Обоснуйте ваш ответ. Приведите примеры использования используя базу данных с уровнем изоляции транзакций Read Committed

Для получения дополнительной информации можно обратиться к фундаментальному труду К. Дж. Дейта, а также к полному руководству по SQL Дж. Гроффа, П. Вайнберга и Э. Опеля.

4. На уровне изоляции транзакций Read Committed имеет место такой феномен, как чтение фантомных строк. Такие строки могут появляться в выборке как в результате добавления новых строк параллельной транзакцией, так и вследствие изменения ею значений атрибутов, участвующих в формировании условия выборки. Рассмотрим пример, иллюстрирующий вторую из указанных причин. На первом терминале организуем транзакцию. Она будет иметь уровень изоляции Read Committed:

```
BEGIN;
BEGIN
SELECT *
FROM aircrafts_tmp
WHERE range > 6000;
aircraft_code | model | range
-----+-----+-----
773 | Boeing 777-300 | 11100
763 | Boeing 767-300 | 7900
319 | Airbus A319-100 | 6700
(3 строки)
```

На втором терминале организуем транзакцию и обновим одну из строк таблицы таким образом, чтобы эта строка стала удовлетворять условию отбора строк, заданному в первой транзакции.

```
BEGIN;  
BEGIN  
UPDATE aircrafts_tmp  
SET range = 6100  
WHERE aircraft_code = '320';  
UPDATE 1
```

Сразу завершим вторую транзакцию, чтобы первая транзакция увидела эти изменения.

```
END;  
COMMIT
```

На первом терминале повторим ту же самую выборку:

```
SELECT *  
FROM aircrafts_tmp  
WHERE range > 6000;  
aircraft_code | model | range  
-----+-----+-----  
773 | Boeing 777-300 | 11100  
763 | Boeing 767-300 | 7900  
319 | Airbus A319-100 | 6700  
320 | Airbus A320-200 | 6100  
(4 строки)
```

Транзакция еще не завершилась, но она уже увидела новую строку, обновленную зафиксированной параллельной транзакцией. Теперь эта строка стала соответствовать условию выборки. Таким образом, не изменяя критерий выборки, мы получили другое множество строк. Завершим теперь и первую транзакцию:

```
END;  
COMMIT
```

Задание. Модифицируйте этот эксперимент: вместо операции UPDATE используйте операцию INSERT. Приведите примеры использования используя базу данных с уровнем изоляции транзакций Read Committed

5. В тексте лекции была рассмотрена команда SELECT ... FOR UPDATE, выполняющая блокировку на уровне отдельных строк. Организуйте две параллельные транзакции с уровнем изоляции Read Committed и выполните с ними ряд экспериментов. В первой транзакции заблокируйте некоторое множество строк, отбираемых с помощью условия WHERE. А во второй транзакции изменяйте условие выборки таким образом, чтобы выбираемое множество строк:

- являлось подмножеством множества строк, выбираемых в первой транзакции;
- являлось надмножеством множества строк, выбираемых в первой транзакции;

- пересекалось с множеством строк, выбираемых в первой транзакции;
- не пересекалось с множеством строк, выбираемых в первой транзакции.

Наблюдайте за поведением команд выборки в каждой транзакции. Попробуйте обобщить ваши наблюдения.

6. Самостоятельно ознакомьтесь с предложением FOR SHARE команды SELECT и выполните необходимые эксперименты. Используйте теорию «Блокировки на уровне строк» и описание команды SELECT. Приведите примеры использования используя базу данных с уровнем изоляции транзакций Read Committed

7. В тексте лекции для иллюстрации изучаемых концепций мы создавали только две параллельные транзакции. Попробуйте воспроизвести представленные эксперименты, создав три или даже четыре параллельные транзакции. Приведите примеры использования используя базу данных с уровнем изоляции транзакций Read Committed

8.* В тексте лекции была рассмотрена транзакция для выполнения бронирования билетов. Для нее был выбран уровень изоляции Read Committed. Как вы думаете, если одновременно будут производиться несколько операций бронирования, то, может быть, имеет смысл «ужесточить» уровень изоляции до Serializable? Или нет необходимости это делать? Обдумайте и вариант с использованием явных блокировок Serializable Обоснуйте ваш ответ.

9.* В теоретическом описании «Уровень изоляции Serializable» сказано, что если поиск в таблице осуществляется последовательно, без использования индекса, тогда на всю таблицу накладывается так называемая предикатная блокировка. Такой подход приводит к увеличению числа сбоев сериализации. В качестве контрмеры можно попытаться использовать индексы. Конечно, если таблица совсем небольшая, то может и не получиться заставить PostgreSQL использовать поиск по индексу. Тем не менее давайте выполним следующий эксперимент. Для его проведения создадим специальную таблицу, в которой будет всего два столбца: один — числовой, а второй — текстовый. Значения во втором столбце будут иметь вид: LOW1, LOW2, ..., HIGH1, HIGH2, ... Назовем эту таблицу modes. Добавим в нее такое число строк, которое сделает очень вероятным использование индекса при выполнении операций обновления строк и, соответственно, отсутствие предикатной блокировки всей таблицы. О том, как узнать, используется ли индекс при выполнении тех или иных операций, написано в литературе и лекции.

```
CREATE TABLE modes AS
```

```
SELECT num::integer, 'LOW' || num::text AS mode
```

```
FROM generate_series( 1, 100000 ) AS gen_ser( num )
```

```
UNION ALL
```

```
SELECT num::integer, 'HIGH' || ( num - 100000 )::text AS mode
```

```
FROM generate_series( 100001, 200000 ) AS gen_ser( num );
```

```
SELECT 200000
```

Проиндексируем таблицу по числовому столбцу.

```
CREATE INDEX modes_ind
```

```
ON modes ( num );
```

```
CREATE INDEX
```

Из всего множества строк нас будут интересовать только две:

```
SELECT *
```

```
FROM modes
```

```
WHERE mode IN ( 'LOW1', 'HIGH1' );
```

```
num | mode
```

```
-----+-----
```

```
1 | LOW1
```

```
100001 | HIGH1
```

(2 строки)

На первом терминале начнем транзакцию и обновим одну строку из тех двух строк, которые были показаны в предыдущем запросе.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN
```

```
UPDATE modes
```

```
SET mode = 'HIGH1'
```

```
WHERE num = 1;
```

```
UPDATE 1
```

На втором терминале тоже начнем транзакцию и обновим другую строку из тех двух строк, которые были показаны выше.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN
```

```
UPDATE modes
```

```
SET mode = 'LOW1'
```

```
WHERE num = 100001;
```

```
UPDATE 1
```

Обратите внимание, что обе команды UPDATE были выполнены, ни одна из них не ожидает завершения другой транзакции.

Попробуем завершить транзакции. Сначала — на первом терминале:

```
COMMIT;
```

```
COMMIT
```

А потом на втором терминале:

```
COMMIT;
```

```
COMMIT
```

Посмотрим, что получилось:

```
SELECT *
```

```
FROM modes
```

```
WHERE mode IN ( 'LOW1', 'HIGH1' );
```

```
num | mode
```

```
-----+-----
```

```
1 | HIGH1
```

```
100001 | LOW1
```

(2 строки)

Теперь система смогла сериализовать параллельные транзакции и зафиксировать их обе. Как вы думаете, почему это удалось? Обосновывая ваш ответ, примите во внимание тот результат, который был бы получен при последовательном выполнении транзакций. с уровнем изоляции Serializable

10.* В тексте лекции был рассмотрен пример транзакции над таблицами базы данных «Авиаперевозки». Давайте теперь создадим две параллельные транзакции и выполним их с уровнем изоляции Serializable. Отправим также двоих пассажиров теми же самыми рейсами, что и ранее, но операции распределим между двумя транзакциями. Отличие заключается в том, что в начале транзакции будут выполняться выборки из таблицы ticket_flights. Для упрощения ситуации не будем предварительно проверять наличие свободных мест, т. к. сейчас для нас важно не это. Итак, первая транзакция:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN
```

```
SELECT *
```

```
FROM ticket_flights
```

```
WHERE flight_id = 13881;
```

```
ticket_no | flight_id | fare_conditions | amount
```

```
-----+-----+-----+-----
```

```
0005433848165 | 13881 | Business | 99800.00
```

```
...
```

```
0005433848007 | 13881 | Economy | 33300.00
```

(82 строки)

```
INSERT INTO bookings ( book_ref, book_date, total_amount )
```

```
VALUES ( 'ABC123', bookings.now(), 0 );
```

```
INSERT 0 1
```

```
INSERT INTO tickets
```

```
( ticket_no, book_ref, passenger_id, passenger_name )
```

```
VALUES ( '9991234567890', 'ABC123', '1234 123456', 'IVAN PETROV' );
```

```
INSERT 0 1
```

```
INSERT INTO ticket_flights
```

```
( ticket_no, flight_id, fare_conditions, amount )
```

```
VALUES ( '9991234567890', 13881, 'Business', 12500 );
```

```
INSERT 0 1
```

```
UPDATE bookings
```

```
SET total_amount = 12500
```

```
WHERE book_ref = 'ABC123';
```

```
UPDATE 1
```

```
COMMIT;
```

```
COMMIT
```

Вторая транзакция:

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
BEGIN
```

```

SELECT *
FROM ticket_flights
WHERE flight_id = 5572;
ticket_no | flight_id | fare_conditions | amount
-----+-----+-----+-----
0005433847924 | 5572 | Business | 99800.00
...
0005433847890 | 5572 | Economy | 33300.00
(100 строк)
INSERT INTO bookings ( book_ref, book_date, total_amount )
VALUES ( 'ABC456', bookings.now(), 0 );
INSERT 0 1
INSERT INTO tickets
( ticket_no, book_ref, passenger_id, passenger_name )
VALUES ( '9991234567891', 'ABC456', '4321 654321', 'PETR IVANOV' );
INSERT 0 1
INSERT INTO ticket_flights
( ticket_no, flight_id, fare_conditions, amount )
VALUES ( '9991234567891', 5572, 'Business', 12500 );
INSERT 0 1
UPDATE bookings
SET total_amount = 12500
WHERE book_ref = 'ABC456';
UPDATE 1
COMMIT;

```

ОШИБКА: не удалось сериализовать доступ из-за зависимостей чтения/записи между транзакциями

ПОДРОБНОСТИ: Reason code: Canceled on identification as a pivot, during commit attempt.

ПОДСКАЗКА: Транзакция может завершиться успешно при следующей попытке.

Задание 10.1. Попробуйте объяснить, почему транзакции не удалось сериализовать. Что можно сделать, чтобы удалось зафиксировать обе транзакции?

Одно из возможных решений — понизить уровень изоляции. Другим решением может быть создание индекса по столбцу `flight_id` для таблицы `ticket_flights`. Почему создание индекса может помочь? Обратитесь за разъяснениями к теоретическому описанию «Уровень изоляции Serializable».

Задание 10.2. В первой транзакции условие в команде `SELECT` такое: ... `WHERE flight_id = 13881`. В команде вставки в таблицу `ticket_flights` значение поля `flight_id` также равно 13881. Во второй транзакции в этих же командах используется значение 5572. Поменяйте местами значения в командах `SELECT` и повторите эксперименты, выполнив транзакции параллельно с уровнем изоляции `Serializable`. Почему сейчас наличие индекса не помогает зафиксировать обе транзакции? Вспомните, что аномалия сериализации —

это ситуация, когда параллельное выполнение транзакций приводит к результату, невозможному ни при каком из вариантов упорядочения этих же транзакций при их последовательном выполнении. Приведите примеры использования используя базу данных с уровнем изоляции Serializable.

Тема 10 Повышение производительности. ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Вопросы для опроса

1. Методы просмотра таблиц.
2. Методы формирования соединений наборов строк.
3. Управление планировщиком.
4. Оптимизация запросов

Перед выполнением упражнений нужно восстановить измененные значения параметров:

SET enable_hashjoin = on;

SET

SET enable_nestloop = on;

SET

1. Как вы думаете, почему при сканировании по индексу оценка стоимости ресурсов, требующихся для выдачи первых результатов, не равна нулю, хотя используется индекс, совпадающий с порядком сортировки?

EXPLAIN

SELECT *

FROM bookings

ORDER BY book_ref;

QUERY PLAN

Index Scan using bookings_pkey on bookings (cost=0.42..8511.24
rows=262788 width=21)
(1 строка)

2. Как вы думаете, если в запросе присутствует предложение ORDER BY, и создан индекс по тем столбцам, которые фигурируют в предложении ORDER BY, то всегда ли будет использоваться этот индекс или нет? Почему? Проверьте ваши предположения с помощью команды EXPLAIN.

Вариант 1 Самостоятельно выполните команду EXPLAIN для запроса, содержащего общее табличное выражение (CTE). Посмотрите, на каком уровне находится узел плана, отвечающий за это выражение, как он оформляется. Учтите, что общие табличные выражения всегда материализуются, т. е. вычисляются однократно и результат их вычисления сохраняется в памяти, а затем все последующие обращения в рамках запроса направляются уже к этому материализованному результату.

4. Прокомментируйте следующий план, попробуйте объяснить значения всех его узлов и параметров.

EXPLAIN

SELECT total_amount

**FROM bookings
ORDER BY total_amount
DESC LIMIT 5;**

QUERY PLAN

```
-----
Limit (cost=8666.69..8666.71 rows=5 width=6)
  -> Sort (cost=8666.69..9323.66 rows=262788 width=6)
        Sort Key: total_amount DESC
        -> Seq Scan on bookings (cost=0.00..4301.88 rows=262788
            width=6)
(4 строки)
```

5. В подавляющем большинстве городов только один аэропорт, но есть и такие города, в которых более одного аэропорта. Давайте их выявим.

**EXPLAIN
SELECT city, count(*)
FROM airports
GROUP BY city
HAVING count(*) > 1;**

QUERY PLAN

```
-----
HashAggregate (cost=3.82..4.83 rows=101 width=25)
  Group Key: city
  Filter: (count(*) > 1)
    -> Seq Scan on airports (cost=0.00..3.04 rows=104 width=17)
(4 строки)
```

Для подсчета числа аэропортов в городах используется последовательное сканирование и формирование хеш-таблицы (HashAggregate), ключом которой является столбец city. Затем из нее отбираются те записи, значения которых соответствуют условию

Filter: (count(*) > 1)

Как вы думаете, чем можно объяснить, что вторая оценка стоимости в параметре cost для узла Seq Scan, равная 3,04, не совпадает с первой оценкой стоимости в параметре cost для узла HashAggregate?

Вариант 2 Выполните команду EXPLAIN для запроса, в котором использована какая-нибудь из оконных функций. Найдите в плане выполнения запроса узел с именем WindowAgg. Попробуйте объяснить, почему он занимает именно этот уровень в плане.

7. Проанализируйте план выполнения операций вставки и удаления строк. Причем сделайте это таким образом, чтобы данные в таблицах фактически изменены не были.

Вариант 3 Замена коррелированного подзапроса соединением таблиц является одним из способов повышения производительности.

Предположим, что мы задались вопросом: сколько маршрутов обслуживают самолеты каждого типа? При этом нужно учитывать, что может иметь место такая ситуация, когда самолеты какого-либо типа не обслуживают ни одного маршрута. Поэтому необходимо использовать не

только представление «Маршруты» (routes), но и таблицу «Самолеты» (aircrafts).

Это первый вариант запроса, в нем используется коррелированный подзапрос.

```
EXPLAIN ANALYZE  
SELECT a.aircraft_code AS a_code, a.model,  
  ( SELECT count( r.aircraft_code )  
    FROM routes r  
   WHERE r.aircraft_code = a.aircraft_code  
 ) AS num_routes  
FROM aircrafts a  
GROUP BY 1, 2  
ORDER BY 3 DESC;
```

А в этом варианте коррелированный подзапрос раскрыт и заменен внешним соединением:

```
EXPLAIN ANALYZE  
SELECT a.aircraft_code AS a_code,  
  a.model, count( r.aircraft_code  
 ) AS num_routes  
FROM aircrafts a  
LEFT OUTER JOIN routes r  
  ON r.aircraft_code = a.aircraft_code  
GROUP BY 1, 2  
ORDER BY 3 DESC;
```

Причина использования внешнего соединения в том, что может найтись модель самолета, не обслуживающая ни одного маршрута, и если не использовать внешнее соединение, она вообще не попадет в результирующую выборку.

Исследуйте планы выполнения обоих запросов. Попробуйте найти объяснение различиям в эффективности их выполнения. Чтобы получить усредненную картину, выполните каждый запрос несколько раз. Поскольку таблицы, участвующие в запросах, небольшие, то различие по абсолютным затратам времени выполнения будет незначительным. Но если бы число строк в таблицах было большим, то экономия ресурсов сервера могла оказаться заметной.

Предложите аналогичную пару запросов к базе данных «Авиаперевозки». Проведите необходимые эксперименты с вашими запросами.

9. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: создание материализованных представлений. В теме 5 было описано такое материализованное представление — «Маршруты» (routes). Команда для его создания была приведена в теме 6.

Проведите эксперимент: сначала выполните выборку из готового представления, а затем ту выборку, которая это представление формирует.

```
EXPLAIN ANALYZE  
SELECT *  
FROM routes;  
EXPLAIN ANALYZE  
WITH f3 AS ( SELECT f2.flight_no, ...
```

Поскольку второй запрос очень громоздкий, то можно поступить таким образом: сначала сохраните его в текстовом файле, а затем выполните с помощью команды \i утилиты psql.

Вы увидите, что затраты времени отличаются практически на два порядка. Конечно, нужно помнить, что материализованные представления необходимо периодически обновлять, чтобы их содержимое было актуальным.

10. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: использование вычисляемых столбцов. Для примера рассмотрим таблицу «Бронирования» (bookings). В ней столбец «Полная сумма бронирования» (total_amount) является вычисляемым. Мы не будем сейчас говорить о том, каким образом его значения синхронизируются с данными в таблице «Перелеты» (ticket_flights), а лишь рассмотрим два запроса, возвращающие полные суммы бронирований. Предположим, что указанного столбца в таблице bookings не было бы. Тогда запрос, возвращающий полные суммы бронирований, выглядел бы так:

```
EXPLAIN ANALYZE  
SELECT b.book_ref, sum( tf.amount )  
FROM bookings b, tickets t, ticket_flights tf  
WHERE b.book_ref = t.book_ref  
AND t.ticket_no = tf.ticket_no  
GROUP BY 1  
ORDER BY 1;
```

Но благодаря наличию вычисляемого столбца total_amount те же сведения можно получить с гораздо меньшими затратами ресурсов:

```
EXPLAIN ANALYZE  
SELECT book_ref, total_amount  
FROM bookings  
ORDER BY 1;
```

Попробуйте предложить еще какой-нибудь вычисляемый столбец для одной из таблиц базы данных «Авиаперевозки». Проведите эксперименты, подтверждающие эффективность вашего решения.

11. Одним из способов повышения производительности является изменение схемы данных, а именно: использование временных таблиц. Предположим, что нам предстоит сделать много выборок из представления «Рейсы» (flights_v), в таком случае имеет смысл подумать о создании временной таблицы:

```
CREATE TEMP TABLE flights_tt AS  
SELECT * FROM flights_v;
```


Сформируйте планы для получения простой выборки из представления и из временной таблицы и сравните полученные результаты. Как вы думаете, почему план, сформированный для получения даже простой выборки из представления, многоуровневый?

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM flights_v;
```

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM flights_tt;
```

Выполните более сложные запросы к представлению и временной таблице и сравните полученные результаты. Чтобы увидеть фактические затраты времени, включайте в команду EXPLAIN опцию ANALYZE.

Подумайте, при выполнении каких запросов к базе данных «Авиаперевозки» было бы целесообразно создать временную таблицу. Создайте ее и проведите эксперименты, подтверждающие эффективность ее использования.

12. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: создание индексов. Выполните следующий простой запрос к таблице «Билеты»:

```
EXPLAIN ANALYZE
```

```
SELECT count( *)
```

```
FROM tickets
```

```
WHERE passenger_name = 'IVAN IVANOV';
```

Создайте индекс по столбцу passenger_name:

```
CREATE INDEX passenger_name_key
```

```
ON tickets ( passenger_name );
```

Теперь повторите запрос и сравните полученные планы и фактические результаты.

Предложите какой-нибудь запрос к базе данных «Авиаперевозки», для выполнения которого было бы целесообразно создать индекс. Создайте индекс и повторите запрос. Изучите полученный план, посмотрите, используется ли индекс планировщиком.

13. В самом конце лекции мы выполняли оптимизацию запроса путем создания индекса и модификации текста запроса. Был сформирован такой запрос:

```
EXPLAIN ANALYZE
```

```
SELECT num_tickets, count( *) AS num_bookings
```

```
FROM
```

```
( SELECT b.book_ref, count( *)
```

```
FROM bookings b, tickets t
```

```
WHERE date_trunc( 'mon', b.book_date ) = '2016-09-01'
```

```
AND t.book_ref = b.book_ref
```

```
GROUP BY b.book_ref
```

```
) AS count_tickets( book_ref, num_tickets )
```

GROUP by num_tickets
ORDER BY num_tickets DESC;

Мы экспериментировали с параметрами планировщика enable_hashjoin и enable_nestloop при наличии индекса по таблице tickets.

SET enable_hashjoin = off;
SET enable_nestloop = off;

Однако полученные планы детально рассмотрены не были.

Задания для самостоятельного выполнения

1. Выполните более сложные запросы к представлению и временной таблице и сравните полученные результаты. Чтобы увидеть фактические затраты времени, включайте в команду EXPLAIN опцию ANALYZE. Подумайте, при выполнении каких запросов к базе данных «Авиаперевозки» было бы целесообразно создать временную таблицу. Создайте ее и проведите эксперименты, подтверждающие эффективность ее использования. Одним из способов повышения производительности является изменение схемы данных, связанное с денормализацией, а именно: создание индексов. Выполните следующий простой запрос к таблице «Билеты»:

```
EXPLAIN ANALYZE
SELECT count( * )
FROM tickets
WHERE passenger_name = 'IVAN IVANOV';
Создайте индекс по столбцу passenger_name:
CREATE INDEX passenger_name_key
ON tickets ( passenger_name );
```

Теперь повторите запрос и сравните полученные планы и фактические результаты. Предложите какой-нибудь запрос к базе данных «Авиаперевозки», для выполнения которого было бы целесообразно создать индекс. Создайте индекс и повторите запрос. Изучите полученный план, посмотрите, используется ли индекс планировщиком.

2.* В самом начале мы выполняли оптимизацию запроса путем создания индекса и модификации текста запроса. Был сформирован такой запрос:

```
EXPLAIN ANALYZE
SELECT num_tickets, count( * ) AS num_bookings
FROM
( SELECT b.book_ref, count( * )
FROM bookings b, tickets t
WHERE date_trunc( 'mon', b.book_date ) = '2016-09-01'
AND t.book_ref = b.book_ref
GROUP BY b.book_ref
) AS count_tickets( book_ref, num_tickets )
GROUP by num_tickets
ORDER BY num_tickets DESC;
```

Мы экспериментировали с параметрами планировщика enable_hashjoin и enable_nestloop при наличии индекса по таблице tickets.

```
SET enable_hashjoin = off;
```

```
SET enable_nestloop = off;
```

Однако полученные планы детально рассмотрены не были.

Задание.2.1 Проанализируйте эти планы. Посмотрите, в каких случаях используются и в каких не используются индексы по таблицам bookings и tickets. Вспомните о таком понятии, как селективность, т. е. доля строк, выбираемых из таблицы. Приведите примеры использования используя базу данных.

В столбцах таблиц могут содержаться значения NULL. При сортировке строк по значениям таких столбцов СУБД по умолчанию ведет себя так, как будто значение NULL превосходит по величине любые другие значения. В результате получается, что если задан возрастающий порядок сортировки, то значения NULL будут идти последними, если же порядок сортировки убывающий, тогда они будут первыми. Принимая решение о создании индексов, нужно учитывать требуемый порядок сортировки и желаемое расположение строк со значениями NULL в выборке (в ее начале или в конце). Давайте создадим таблицу, содержащую такое число строк, что использование индекса планировщиком становится очень вероятным, и выполним с этой таблицей ряд экспериментов.

```
CREATE TABLE nulls AS
```

```
SELECT num::integer, 'TEXT' || num::text AS txt
```

```
FROM generate_series( 1, 200000 ) AS gen_ser( num );
```

```
SELECT 200000
```

Проиндексируем таблицу по числовому столбцу. Поскольку мы не указываем порядок сортировки явным образом, то по умолчанию он будет возрастающим, как если бы мы использовали ключевое слово ASC.

```
CREATE INDEX nulls_ind
```

```
ON nulls ( num );
```

```
CREATE INDEX
```

Добавим в таблицу одну строку, содержащую значение NULL в индексируемом столбце:

```
INSERT INTO nulls
```

```
VALUES ( NULL, 'TEXT' );
```

```
INSERT 0 1
```

Теперь посмотрим, будет ли использоваться индекс в следующем запросе:

```
EXPLAIN
```

```
SELECT *
```

```
FROM nulls
```

```
ORDER BY num;
```

Да, индекс используется.

```
QUERY PLAN
```

```
-----  
Index Scan using nulls_ind on nulls (cost=0.42..5852.42 rows=200000  
width=14)  
(1 строка)
```

Вы можете убедиться, что строка со значением NULL окажется в выводе самой последней. Поскольку в нашей таблице очень много строк, воспользуемся предложением OFFSET:

```
SELECT *  
FROM nulls  
ORDER BY num  
OFFSET 199995;  
num | txt  
-----+-----  
199996 | TEXT199996  
199997 | TEXT199997  
199998 | TEXT199998  
199999 | TEXT199999  
200000 | TEXT200000  
| TEXT  
(6 строк)
```

Модифицируем запрос, явно указав, что значения NULL должны располагаться в самом начале выборки, и посмотрим, будет ли использоваться индекс теперь.

```
EXPLAIN  
SELECT *  
FROM nulls  
ORDER BY num NULLS FIRST;
```

Теперь индекс не используется. Вместо этого выполняется последовательное сканирование таблицы и сортировка выбранных строк.

```
QUERY PLAN
```

```
-----  
Sort (cost=24110.14..24610.14 rows=200000 width=14)  
Sort Key: num NULLS FIRST  
-> Seq Scan on nulls (cost=0.00..3081.00 rows=200000 width=14)  
(3 строки)
```

Задание. 2.2. Ниже приведена модифицированная команды выборки из таблицы nulls. Не выполняя эту команду, попытайтесь ответить, будет ли использоваться созданный нами индекс при выполнении такой выборки, а затем проверьте вашу гипотезу на практике.

```
EXPLAIN  
SELECT *  
FROM nulls  
ORDER BY num DESC NULLS FIRST;
```

Обратите внимание на ключевое слово Backward в плане выполнения запроса. Что оно означает?

Задание 2.3. Модифицируйте команду создания индекса таким образом, чтобы он использовался при выполнении следующей выборки:

```
SELECT *  
FROM nulls  
ORDER BY num NULLS FIRST;
```

Задание 2.4. Выполните аналогичные эксперименты, задавая убывающий порядок сортировки с помощью ключевого слова DESC и изменяя расположение значений NULL в выборке с помощью ключевых слов NULLS FIRST и NULLS LAST предложения ORDER BY. С помощью команды EXPLAIN ANALYZE посмотрите, каким будет фактическое время выполнения команд. За дополнительной информацией обратитесь к описанию команды CREATE INDEX, приведенному в документации.

3. Обратитесь к запросам. Выполните команду EXPLAIN для всех этих запросов и ознакомьтесь с планами, которые создаст планировщик. В планах могут встречаться наименования методов, которые не были рассмотрены, однако они должны быть вам интуитивно понятны.

4. В лекции «Планирование запросов» приведены параметры, с помощью которых можно влиять на решения, принимаемые планировщиком. В тексте лекции уже говорили о параметрах, управляющих выбором способа соединения наборов строк, и показали простой пример. Также было сказано и о том, что при установке значений параметров enable_hashjoin, enable_mergejoin и enable_nestloop в off не накладывается полного запрета на использование соответствующих методов. Вместо этого конкретному методу назначается очень высокая стоимость. Приведите самостоятельно примеры реализации из базы данных.

Давайте проведем следующий эксперимент: запретим использование всех методов соединения наборов строк и выполним запрос, в котором соединяются две таблицы.

```
SET enable_hashjoin = off;  
SET enable_mergejoin = off;  
SET enable_nestloop = off;  
Запрос выводит информацию о числе мест в самолетах всех моделей.  
EXPLAIN  
SELECT a.model, count( * )  
FROM aircrafts a, seats s  
WHERE a.aircraft_code = s.aircraft_code  
GROUP BY a.aircraft_code;  
QUERY PLAN  
-----  
GroupAggregate      (cost=100000000000.41..100000000109.95      rows=9  
width=56)  
Group Key: a.aircraft_code  
-> Nested Loop  
(cost=100000000000.41..100000000103.16 rows=1339 width=48)  
-> Index Scan using aircrafts_pkey on aircrafts a
```

```
(cost=0.14..12.27 rows=9 width=48)
-> Index Only Scan using seats_pkey on seats s
(cost=0.28..8.61 rows=149 width=4)
Index Cond: (aircraft_code = a.aircraft_code)
(6 строк)
```

Обратите внимание на оценки стоимости выполнения запроса. Резкое повышение оценок происходит именно в узле, отвечающем за соединение наборов строк. Эти оценки не означают, что время выполнения запроса будет стремиться к бесконечности. С помощью команды EXPLAIN ANALYZE выполните запрос и убедитесь в этом сами.

Задание. Самостоятельно ознакомьтесь с содержанием раздела «Планирование запросов», а также раздела «Управление планировщиком с помощью явных предложений JOIN» и проведите эксперименты с запросами, и получая различные варианты планов и сравнивая их.

Ваша задача — понять, как изменения значений этих параметров влияют на план выполнения запроса, написать собственные примеры-решения используя базу данных. Однако для того чтобы понимать, когда и почему нужно изменять значения конкретных параметров, правильно оценивать степень и направленность их влияния, понимать взаимосвязь параметров, требуется опыт и изучение документации.

5. Самостоятельно ознакомьтесь с разделом «Статистика, используемая планировщиком» и привести примеры использования используя базу данных.

6. Команда EXPLAIN имеет опцию BUFFERS. Ознакомьтесь с ней самостоятельно по разделу «Использование EXPLAIN» и привести примеры использования используя базу данных.

7. При массовом вводе данных в базу данных производительность СУБД может снижаться по ряду причин, например, при наличии индексов они обновляются при вводе каждой новой строки в таблицу, а это требует дополнительных затрат ресурсов. Для повышения производительности СУБД в подобных ситуациях в документации предлагается ряд мер, например, удаление индексов перед началом массового ввода данных и пересоздание индексов после завершения такого ввода. Ознакомьтесь с этими мерами самостоятельно по разделу «Наполнение базы данных». Смоделируйте ситуации, описанные в этом разделе документации, и выполните рекомендуемые действия используя базу данных.

Тема 11 Объектно-ориентированные базы данных ОПК-2.2, ОПК ОС-10.2, ОПК ОС-11.1

Вопросы для опроса

1. Понятие объектно-ориентированных баз данных (ООБД) и объектно-ориентированных систем управления базами данных (ООСУБД).
2. Характеристики ООБД.
3. Достоинства и недостатки модели ООБД.

Контрольные задания с развернутым ответом

Практическая работа Связь СУБД PostgreSQL и Python

Цель работы: произвести связь базы данных в PostgreSQL и Python, изучить операции по манипулированию с данными БД, а также созданию простейших пользовательских функций.

Библиотеки, которые нужны в Python:

- psycopg2 для соединения с БД
- pandas для работы с данными

По желанию вы можете использовать другие библиотеки.

1 Соединение Python с БД в PostgreSQL

Стандартный вариант соединения Python с БД в PostgreSQL:

```
import pandas as pd import psycopg2
```

```
# Подключение к базе данных:
```

```
connection = psycopg2.connect(database="database1", # название базы  
данных  
user="postgres",  
password="admin",  
host="127.0.0.1",  
port="5432")
```

С помощью метода connect() создается подключение к экземпляру базы данных

PostgreSQL. Здесь

- database – название БД, к которой нужно подключиться,
- user – имя пользователя для аутентификации,
- password – пароль БД,
- host – адрес сервера БД,
- port – номер порта (значение по умолчанию 5432).

С базой данных можно взаимодействовать с помощью класса cursor. Его можно

получить из метода cursor(), который есть у объекта соединения. Он поможет выполнять SQL-команды из Python.

Из одного объекта соединения можно создавать неограниченное количество объектов cursor. Они не изолированы, поэтому любые изменения, сделанные в базе данных с помощью одного объекта, будут видны остальным.

С помощью connection.get_dsn_parameters() можно вывести параметры соединения.

С помощью метода execute объекта cursor можно выполнить любую операцию или запрос к базе данных. В качестве параметра этот метод принимает SQL-запрос. Результаты запроса можно получить с помощью fetchone(), fetchmany(), fetchall().

Необходимо использовать cursor.close() и connection.close(), так как правильно всегда закрывать объекты cursor и connection после завершения работы, чтобы избежать проблем с базой данных.

Выведем данные о соединении и версии СУБД:

```
cursor = connection.cursor()# курсор для выполнения операций с БД
```

```
print(connection.get_dsn_parameters(), "\n") # вывод свойства соединения
cursor.execute("SELECT version();") # выполнение запроса к БД
version_ps = cursor.fetchone() # получение результата запроса print("Вы
подключены - ", version_ps, "\n")
```

Результат:

```
{'user': 'postgres', 'channel_binding': 'prefer', 'dbname': 'database1', 'host':
'127.0.0.1', 'port': '5432', 'options': '', 'sslmode': 'prefer', 'sslcompression': '0', 'sslsni':
'1', 'ssl_min_protocol_version': 'TLSv1.2', 'gssencmode': 'disable', 'krbsrvname':
'postgres', 'target_session_attrs': 'any'}
```

Вы подключены к - ('PostgreSQL 13.5, compiled by Visual C++ build 1914, 64-bit',)

2 Выполнение запроса SELECT

Выполним запрос на выборку данных из таблицы с работами: sql = "SELECT * FROM jobs " # запрос SQL

```
df = pd.read_sql_query("SELECT * FROM jobs ", connection) print(df)
```

В данном примере использовали read_sql_query библиотеки pandas, который возвращает DataFrame, соответствующий результирующему набору строки запроса.

3 Создание новой таблицы с помощью Python и заполнение ее данными

Создадим новую таблицу locations

```
cursor.execute("CREATE TABLE if not exists locations (location_id int
PRIMARY KEY, city varchar(30),
postal_code varchar(12) ); ")
```

Проверим, что таблица появилась в СУБД PostgreSQL (рисунок 1):



Рисунок 1 – Создание новой таблицы

Заполним таблицу данными и выведем результат:

```
cursor.execute(
"INSERT INTO locations VALUES (1, 'Roma', '00989')
")
"
```

```
connection.commit()
```

```
table_locations = pd.read_sql_query("SELECT * FROM locations ",
connection)
```

```
print(table_locations)
```

Проверим, что таблица также заполнена в СУБД PostgreSQL (рисунок 2):

	location_id	city	postal_code
1	1	Roma	00989

Рисунок 2 – Заполнение таблицы

4 Создание простейших пользовательских функций в PostgreSQL и Python

Создадим функцию select_data в PostgreSQL, которая на вход получает код отдела и выводит информацию из таблицы departments, фильтруя данные по коду отдела.

```
CREATE FUNCTION select_data(id_dept int) RETURNS
SETOF departments AS $$ SELECT * FROM departments WHERE departmen
ts.department_id > id_dept;
```

```
$$ LANGUAGE SQL;
```

```
SELECT * FROM select_data(30);
```

Результат вызова на рисунке 3 (выведены только отделы с id>30):

department_id	department_name	manager_id
50	Human Resources	53
50	Shipping	22
80	IT	4
80	Sales	48
90	Executive	1
100	Finance	9

Рисунок 3 – Создание функции

Теперь вернемся в Python и попробуем вызвать созданную функцию из скрипта в Python:

```
cursor.callproc('select_data',[20,])# вызов функции (название
изPostgreSQL )
```

```
result = cursor.fetchall() # получение результатов
```

```
result_proc = pd.DataFrame(result)# создание датафрейма с результатом
print(result_proc)
```

Попробуем выполнить обратное действие, создадим функцию select_data1, аналогичную функции выше в скрипте Python и внесем изменения в БД в PostgreSQL:

```
| #код функции
```

```
postgresql_func = """
```

```
CREATE OR REPLACE FUNCTION select_data1(id_dept int) RETURNS
SETOF departments AS $$
```

```
SELECT * FROM departments WHERE departments.department_id >
id_dept;
```

```
$$ LANGUAGE SQL;
```

```
"""
```

```
cursor.execute(postgresql_func) # выполнение запроса
```

```
connection.commit()
```

нен

#внесениеизме

ийвБД

connectionзакрытсоед.close()и
нения #

cursorзакрытие.cursorlo() #

Проверим, как это выглядит в нашей СУБД (рис.4):

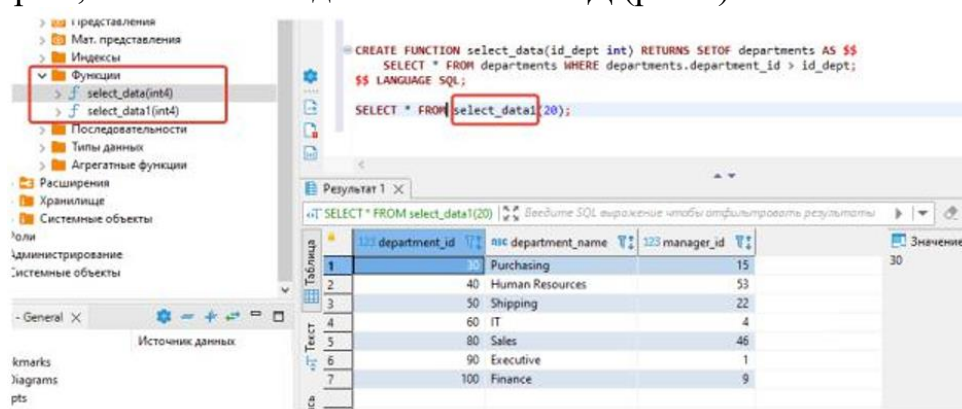


Рисунок 4 - Результат

Порядок выполнения работы:

- 1.Использовать БД, созданную в предыдущей ЛР (employees, departments, jobs). Обязательно предоставить схему данных в отчете.
- 2.Осуществить связь Python и БД в PostgreSQL.
- 3.Создать новую таблицу locations, по примеру методических указаний. Заполнить таблицу данными:

```
INSERT INTO locations VALUES ( 1,'Roma', '00989'); INSERT INTO lo  
cations VALUES ( 2,'Venice', '10934'); INSERT INTO locations VALUES ( 3,'To  
kyo', '1689'); INSERT INTO locations VALUES ( 4,'Hiroshima', '6823'); INSERT  
INTO locations VALUES ( 5,'Southlake', '26192');
```

```
INSERT INTO locations VALUES ( 6,'South San  
Francisco', '99236'); INSERT INTO locations VALUES ( 7,'South  
Brunswick', '50090'); INSERT INTO locations VALUES ( 8,'Seattle', '98199');  
INSERT INTO locations VALUES ( 9,'Toronto', 'M5V 2L7'); INSERT  
INTO locations VALUES ( 10,'Whitehorse', 'YSW 9T2');
```

- 4.Добавить в таблицу с сотрудниками location_id, добавить связь с таблицей locations, заполнить столбец location_id данными. В отчете описать, каким способом было выполнено задание, приложить скриншоты результатов, в том числе обновленную схему данных.

- 5.Выполнить 3 запроса SELECT в Python, предоставить скриншоты результатов и текстовое пояснение:

6. Создать функцию select_data в СУБД PostgreSQL (см. Методические указания 4 пункт), продемонстрировать результат работы. Вызвать функцию select_data в Python, продемонстрировать результат. Создать функцию select_data1 в скрипте Python, продемонстрировать результат вызова этой функции в СУБД PostgreSQL.

Контрольное задание

Создать собственную пользовательскую функцию в СУБД PostgreSQL, используя более сложные SQL-запросы (например, с применением

агрегатных функций и связью нескольких таблиц), продемонстрировать результат работы. Создать эту же функцию через скрипт Python.

Таблица 1 - Варианты заданий

Варианты:	Формулировка запросов
1	- Найти сотрудника, фамилия которого состоит более чем из одного слова. - Найти минимальные зарплаты по отделам, но вывести только те, которые больше 10000.
2	- Вывести количество сотрудников в каждом отделе и отсортировать по убыванию числа сотрудников. - Найти сотрудника, работающего программистом (job_id = 'IT_PROG') и имеющего самую высокую зарплату среди коллег.
3	- Найти сотрудника с именем John, имеющего зарплату больше 12000. - Вывести название отдела с максимальным числом сотрудников.
4	- Найти средние зарплаты по каждому из отделов (можно ли как-то округлить полученные значения?). - Вывести названия должностей и количество сотрудников, соответствующих им. Отсортировать данные по убыванию числа сотрудников.
5	- Найти сотрудника, который имеет зарплату равную минимальной зарплате по его должности. - Найти сотрудника с минимальной зарплатой среди всех сотрудников. Имя и фамилию вывести в одной колонке, дать имя колонке «worker».
6	- Найти первых трёх сотрудников с наименьшей разницей между их зарплатой и минимальной зарплатой по должности. - Найти самое популярное имя (first_name).
7	- Найти сотрудника со второй по счёту минимальной зарплатой. - Выяснить, сколько фондовых менеджеров (Stock_manager) работают в отделе перевозок (Shipping).
8	- Найти количество сотрудников, в должности которых фигурирует слово «Manager». - Найти разницу между зарплатой начальников и средней зарплатой их подчиненных.
9	- Выяснить, сотрудники какой профессии получают зарплату меньше 2500. - Вывести сотрудников и их начальников (в одном столбце расположены сотрудники, во втором – их начальники).
10	- Найти профессию, диапазон которой между минимальной и максимальной зарплатой меньше, чем у остальных профессий. - Вывести названия профессий(job_title) и среднюю зарплату (в диапазоне от 2000 до 5000) сотрудников этих профессий.

Тема 12 Использование XML при работе с БД. ОПК-2.2, ОПК ОС-11.1

Вопросы для опроса

1. XML-ориентированные базы данных.
2. Типы XML-данных в SQL Server.
3. Импорт и экспорт XML-документов в SQL Server.

Контрольные задания с развернутым ответом

Практическая работа. Работа с данными в формате XML

Практическая работа посвящена вопросам обработки данных в формате XML средствами, предоставляемыми СУБД SQL Server. При выполнении этой работы будет использоваться БД ExampleBase, файл с резервной копией которой находится в папке с файлами для данной работы. Получить из резервной копии базу данных можно из Management Studio, выбрав пункт "Восстановить базу данных" в контекстном меню узла "Базы данных" (рис. 1) При необходимости воспользуйтесь справочной системой.

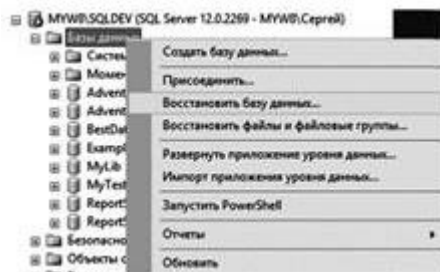


Рис. 1. Восстановление БД

База содержит две таблицы, в первой из которых (Country) – информация о странах, во второй – о регионах (Region). На рис. 2 представлена диаграмма с изображением этих таблиц.



Рис. 2. Диаграмма с таблицами базы ExampleBase

Запустите приложение Management Studio, подключитесь к экземпляру SQL Server и восстановите базу ExampleBase из резервной копии. Ознакомьтесь с содержимым таблиц базы данных. Выполните следующие задания.

Задание 1.

Напишите запрос, выводящий из таблицы Country данные о буквенном коде страны (ID), ее сокращенном и полном названии, столице в виде XML. При этом, корневой элемент должен называться Countries, элемент следующего уровня – Country. Значения столбцов таблицы выводятся как текстовые данные элементов, названия которых соответствуют названиям столбцов. Фрагмент подобного XML-документа приведен ниже:

```
<Countries>
  <Country>
```

```
<ID> AUS</ID>
<NAME> Австралия </NAME>
<FULLNAME> Австралийский Союз </FULLNAME>
<CAPITAL> Канберра </CAPITAL>
</Country>
</Countries>
```

Задание 2

Создайте временную таблицу #T1 со столбцами код страны (первичный ключ), полное и сокращенное название страны, название столицы, перечень регионов (в виде xml-документа). Сначала заполните в ней столбцы, кроме последнего, данными из таблицы Country. Вторым выражением SQL заполните значение последнего столбца (xml), записав туда документ, содержащий перечень регионов соответствующей страны. Название региона оформляйте как текстовые данные элемента Region, код региона (REGID) и название столицы региона оформите как атрибуты соответствующего элемента.

Задание 3

Используя данные из таблицы #T1 и возможности языков SQL и XQuery, напишите запросы, выводящие в реляционном представлении (в виде таблицы):

- название страны и региона, административный центр которого город BELFAST;
- перечень регионов и их административные центры в Австралии (полное название Австралийский Союз) и Великобритании (Соединенное Королевство Великобритании и Северной Ирландии)